

Real-Time Linear Predictive Filter with GUI Documentation

Xhovani Mali
ECE-GY 6183 DSP Lab
NYU Tandon School of Engineering

Fall 2024

Overview

This project implements a real-time financial signal analyzer using Linear Predictive Coding (LPC). It predicts future values of financial time-series data and visualizes actual vs. predicted data in real-time. The system integrates dynamic graphical user interfaces (GUIs) and provides real-time control over LPC parameters.

TurboLPC Integration

TurboLPC is a fast, simple yet powerful Python library that provides the functionality of Linear Predictive Coding (LPC) for signals. It is engineered for performance, offering LPC modeling functions that are significantly faster—by orders of magnitude—than alternative libraries such as Spectrum.

What is Linear Predictive Coding?

LPC is a method used in signal processing to estimate the stochastic random process of a signal. It models the spectral envelope of a signal and predicts future signal values based on past values, making it useful for:

- Speech and audio analysis
- Data compression
- Feature extraction

TurboLPC Features and Capabilities

- **Fast LPC Modeling:** Quickly calculate LPC coefficients for time series data using Burg's method.
- **Frequency-Warped LPC Modeling:** Apply frequency-warping in LPC modeling for dynamic spectral resolution.
- **Cepstrum Coefficients:** Easily convert between LPC model coefficients and cepstral coefficients.
- **Time Series Generation:** Generate time series data from LPC model coefficients with ease.
- **Power Spectrum Density Estimation:** Obtain precise LPC model-based power spectrum density estimates.

Why TurboLPC for This Project?

- **Exceptional Speed:** TurboLPC handles larger datasets and complex computations efficiently, critical for real-time processing.
- **Advanced Frequency-Warping Capabilities:** Its frequency-warped LPC feature enables more accurate spectral analysis by addressing the non-uniform distribution of useful information in time series data.

Original Contributions and Distinction from TurboLPC

While TurboLPC is used for core LPC modeling, this project extends its capabilities through:

- Real-time integration with financial data simulations generated via Geometric Brownian Motion (GBM).
- Dynamic GUI interfaces for adjusting LPC parameters in real-time.
- Visualization modules for plotting raw and predicted financial data.
- Export functionality for analytical outputs, implemented via separate modules.

Code written for this project is maintained in a distinct folder and complements the TurboLPC library's features to provide an end-to-end solution tailored to financial signal analysis.

Features

- Real-time stock price simulation using Geometric Brownian Motion (GBM).
- Predictive modeling using LPC filtering.
- Visualization of raw and filtered stock price data with real-time predictive success metrics.
- Interactive GUI with sliders for LPC parameters.
- Data export functionality for analysis in Excel (currently non-functional due to NaN values generated by the `analysis.py` package).

Function Headers

Analysis Functions

```
def arburg_vector(x, order=1):  
    """Calculates LPC coefficients for a single time series."""  
  
def arburg_matrix(X, order=1):  
    """Calculates LPC coefficients for multiple time series."""  
  
def arburg_warped_vector(x, order=1, warp_factor=0):  
    """Calculates frequency-warped LPC coefficients for a single time series."""
```

Utility Functions

```
def set_smooth_ylim(ax, all_prices, margin=5):  
    """Dynamically adjusts Y-axis limits."""  
  
def apply_decay(coeffs, decay_factor):  
    """Applies exponential decay to LPC coefficients."""
```

Real-Time Plotting Functions

```
def real_time_plot(lpc_order, window_size, decay_factor, fig, ax, canvas,  
                  plot_running, raw_prices, filtered_prices):  
    """Plots stock price data and LPC-filtered predictions in real-time."""
```

Synthesis Functions

```
def gen_ts(a, sigma=1, n_samples=1000, data=None):  
    """Generates time series data using LPC coefficients."""
```

Conversion Functions

```
def arcoeff_to_cep(a, sigma_squared, N):  
    """Converts LPC coefficients to cepstral coefficients."""  
  
def cep_to_arcoeff(c, order):  
    """Converts cepstral coefficients to LPC coefficients."""
```

Frequency Response Functions

```
def freqz(a=1, sigma_squared=1, worN=1000, whole=False, fs=500):  
    """Calculates the frequency response of the LPC model."""
```

Warping Functions

```
def arcoeff_warp(a, warp_factor, task="warp"):  
    """Warps or unwarps LPC coefficients."""
```

Implementation Details

1. Data Simulation

Stock price data is generated in real-time using the Geometric Brownian Motion (GBM) model implemented in `stock_simulation.py`.

2. Signal Processing

LPC-based signal prediction is performed using functions from `analysis.py`, including:

- `arburg_vector()` for single signal LPC analysis.
- `arburg_matrix()` for LPC analysis across multiple signals.
- Frequency-warped LPC is supported via `arburg_warped_vector()` and `arburg_warped_matrix()`.

3. Visualization

The `plot.py` module provides real-time visualization, featuring:

- Raw stock price data (blue line).
- LPC-filtered predictions (red line).
- Predictive success rate displayed dynamically in the plot title.
- Real-time adaptive Y-axis scaling using `set_smooth_ylim()`.

4. Interactive GUI

The graphical user interface is built using Tkinter (`gui.py`) and includes:

- Sliders for adjusting LPC order, window size, and decay factor.
- Start/Stop buttons for simulation and plotting.
- Save Data button to export data to Excel using `convert_to_excel.py`.

5. Utility Functions

Utility functions from `utils.py` include:

- `arcoeff_to_cep()` for LPC-to-cepstral coefficient conversion.
- `cep_to_arcoeff()` for cepstral-to-LPC coefficient conversion.
- `freqz()` for frequency response calculation.
- `arcoeff_warp()` for LPC coefficient warping.

Performance Enhancements and Improvements

This project includes several performance optimizations to ensure smooth real-time plotting and responsive GUI interactions:

1. Plotting Performance

To improve the interaction between Tkinter and Matplotlib threading, an FPS sleep timer was introduced, capping the frame rate at 30 frames per second. This adjustment reduces the computational load, ensuring smoother performance and preventing GUI freezes during real-time plotting.

2. Responsive Variable Adjustments

A mechanism was added to automatically stop the ongoing plot when LPC parameters (e.g., LPC order, window size, or decay factor) are adjusted using the sliders. This ensures that:

- The system applies new parameters without conflicting with an active plotting thread.
- Users experience a responsive and error-free interface.

3. Multi-threading Improvements

The `CreateGUI` function uses a separate thread to handle real-time plotting (`real_time_plot`) while maintaining responsiveness in the GUI. Key updates include:

- Explicit control of the `plot_running` flag to manage the lifecycle of the plotting thread.
- Resetting price lists (`raw_prices` and `filtered_prices`) only when the plotting thread starts, ensuring consistent data handling during successive sessions.

Code Changes Summary

Key additions to the implementation:

- A 30 FPS limit added to the `real_time_plot` function.
- A `slider_changed` function was introduced, which stops the ongoing plot when sliders are adjusted.
- Improved threading logic to prevent redundant data clearing during restarts.

Screenshots and Examples

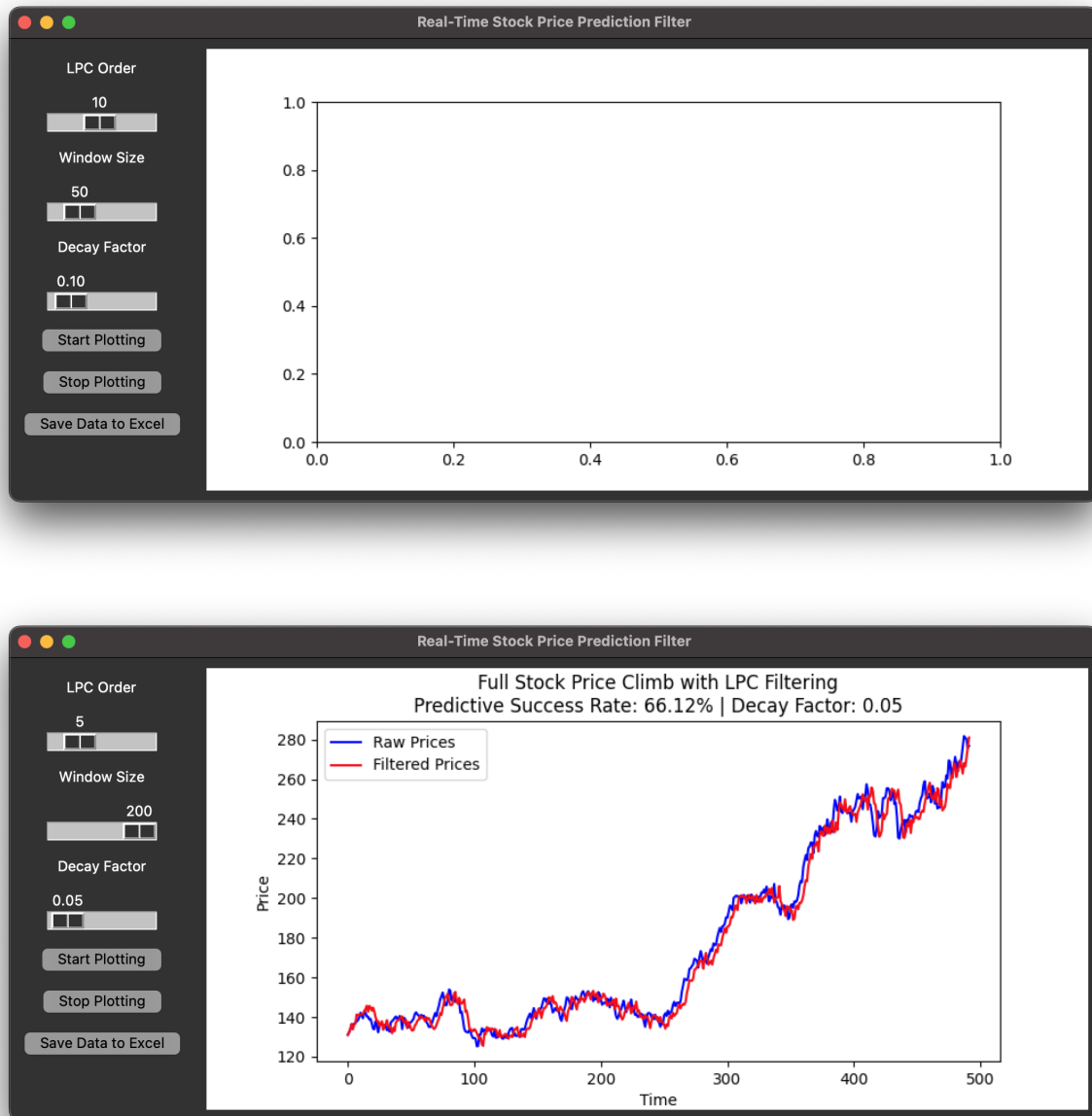


Figure 1: Real-time visualization of actual vs. predicted data.

Usage Instructions

1. Clone this repository:

```
git clone https://github.com/XhovaniM8/Real-Time-Stock-Prediction-Filter
cd real-time-stock-prediction-filter
```

2. Set Up a Python Virtual Environment (Optional):

```
python3 -m venv .venv
source .venv/bin/activate # On Windows, use .venv\Scripts\activate
```

3. Install Dependencies:

```
pip install -r requirements.txt
```

4. Run the application:

```
python src/main.py
```

5. Use the GUI to start/stop processing, adjust LPC parameters, and export data.

Conclusion

This project demonstrates the integration of DSP techniques for the analysis of real-time financial data. The combination of LPC-based modeling, interactive GUI, and advanced visualization extends the implementation beyond class concepts, providing a robust tool for exploring financial signal dynamics.

References

- Python Libraries Documentation: <https://docs.python.org/3/library/>
- TurboPLC Github Source & Documentation: <https://github.com/MDFahimAnjum/TurboLPC>
- Simulating Brownian Motion (and Stock Prices) using Python: <https://shorturl.at/as7bA>